# What Person C owns

You're the **backbone** of the system — everything that isn't intake understanding (A) or knowledge/retrieval (B):

1. **Orchestrator** (LangGraph) — the router that decides which node runs next based on completeness score, deflection confidence, duplicate match, and classification confidence
2. **Classify & Route agent** — few-shot LLM classification across your 5 categories, with a confidence score attached
3. **Frontend** — single-page chat UI (input → clarify Q&A → deflect offer → ticket preview)
4. **Integration** — wiring all 4 nodes (yours + A's two + B's two) into one working LangGraph graph
5. **Audit logging** — SQLite writes at every gate (completeness gate, deflection gate, duplicate gate, confidence gate)

You're the only role touching *every* part of the pipeline, since orchestration and integration require your hands in everyone else's nodes.

# Day 1 (today, Aug 28 evening → Aug 29) — Your specific tasks

While A and B are heads-down generating synthetic tickets for their categories, you work in parallel on scaffolding:

**Step 1 — Repo + FastAPI skeleton**

- Stand up a FastAPI project with a single `/chat` endpoint (or WebSocket if you want live back-and-forth — optional for Day 1, can be plain POST for now)
- Get it running locally, returning a stub JSON response

**Step 2 — LangGraph skeleton with 4 dummy nodes + router**

- Define your `StateGraph` with the shared state object (session_id, user_messages, extracted_fields, missing_fields, completeness_score, retrieved_kb_docs, similar_tickets, classification_confidence, routing_decision, escalation_needed, audit_log)
- Create 4 placeholder nodes: `intake_clarify`, `deflect`, `duplicate_check`, `classify_route` — each just returns a hardcoded/stub output for now
- Wire a basic router function between them (even if the branching logic is fake/always-true for now) — the point is proving the graph *compiles and runs end-to-end*, not that it's smart yet

**Step 3 — Wire up Chroma and SQLite, empty**

- Instantiate an empty ChromaDB persisted collection (so B can start embedding into it Day 2 without setup friction)
- Create your SQLite schema for the audit log (session_id, stage, decision, confidence, timestamp) and confirm you can write/read a dummy row

**End of Day 1 goal:** the skeleton runs start-to-finish with stub logic — not smart, just plumbed. This unblocks A and B to plug real logic into your nodes on Day 2 without touching the graph structure themselves.

# Quick sanity checklist before you start coding

- Confirm the 5 categories and their completeness checklists are finalized (30-min sync with A and B first — this affects your classify prompt and A's clarify logic)
- Decide now whether chat is WebSocket or plain POST — don't decide this later, it changes your frontend and backend shape
- Get Azure OpenAI credentials/endpoint working in a throwaway script before building it into FastAPI, so you're not debugging auth *and* orchestration logic at the same time

Want me to sketch the actual LangGraph state schema and node/router skeleton code to get Step 2 moving?

Here's a concrete path to get from zero to a running skeleton today.

# Software you need installed

| Tool | Purpose | Check/Install |
|---|---|---|
| Python 3.11 | Runtime | `python3.11 --version` |
| pip | Package manager | comes with Python |
| Git | Version control | `git --version` |
| Node.js (18+) | Only if you go React; skip if plain HTML/JS | `node -v` |
| VS Code (or any editor) | — | — |
| Azure OpenAI access | LLM calls | endpoint + API key + deployment name |
| curl or Postman | Testing endpoints without a frontend first | — |

For "most basic frontend," skip React/Next.js entirely — a single `index.html` with vanilla JS `fetch()` calls is faster to stand up and perfectly fine for a 4-day demo. You can always upgrade later if time allows.

# Step 1 — Virtual environment

mkdir tro-backend && cd tro-backend
python3.11 -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
python -m pip install --upgrade pip

# Step 2 — Install dependencies

pip install fastapi uvicorn[standard] langgraph langchain-openai python-dotenv chromadb sentence-transformers pydantic

- `fastapi` + `uvicorn` — backend server
- `langgraph` + `langchain-openai` — orchestration + Azure OpenAI calls
- `chromadb` — vector store
- `sentence-transformers` — embeddings (all-MiniLM)
- `python-dotenv` — load Azure credentials from `.env`
- `pydantic` — state schema

SQLite needs nothing extra — it's in the Python standard library (`sqlite3`).

Freeze it immediately so A and B can replicate your environment:

pip freeze > requirements.txt

# Step 3 — Project structure

```
tro-backend/
├── venv/
├── .env
├── requirements.txt
├── main.py              # FastAPI app
├── graph.py             # LangGraph skeleton
├── state.py             # shared state schema
├── db.py                # SQLite audit log
├── chroma_store.py      # Chroma init
├── nodes/
│   ├── intake_clarify.py   # stub for Person A
│   ├── deflect.py          # stub for Person B
│   ├── duplicate_check.py  # stub for Person B
```

```
│   └── classify_route.py   # yours
└── frontend/
    └── index.html
```

## Step 4 — `.env`

```
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/
AZURE_OPENAI_API_KEY=xxxx
AZURE_OPENAI_DEPLOYMENT=your-deployment-name
AZURE_OPENAI_API_VERSION=2024-02-15-preview
```

Test this works in isolation before touching FastAPI — a 5-line throwaway script that just calls the LLM and prints a response. Don't debug auth and orchestration at the same time.

## Step 5 — `state.py` (shared state object)

```
from typing import TypedDict, Optional, List, Dict

class TicketState(TypedDict):
    session_id: str
    user_messages: List[str]
    extracted_fields: Dict
    missing_fields: List[str]
    completeness_score: float
    retrieved_kb_docs: List[Dict]
    similar_tickets: List[Dict]
    classification_confidence: float
    routing_decision: Optional[str]
    escalation_needed: bool
    audit_log: List[Dict]
```

## Step 6 — Stub nodes (one file each, dummy logic for Day 1)

```
# nodes/intake_clarify.py
def intake_clarify(state):
    state["completeness_score"] = 0.5
    state["missing_fields"] = ["urgency"]
    return state
```

Repeat the same pattern for `deflect.py`, `duplicate_check.py`, `classify_route.py` — each just mutates a placeholder field and returns state. A and B will replace the internals later; you're not touching their logic, just proving the plumbing.

## Step 7 — `graph.py` (LangGraph skeleton + router)

```
from langgraph.graph import StateGraph, END
from state import TicketState
from nodes.intake_clarify import intake_clarify
from nodes.deflect import deflect
from nodes.duplicate_check import duplicate_check
from nodes.classify_route import classify_route

def router(state):
    if state["completeness_score"] < 0.7:
        return "intake_clarify"
    return "deflect"

builder = StateGraph(TicketState)
builder.add_node("intake_clarify", intake_clarify)
builder.add_node("deflect", deflect)
builder.add_node("duplicate_check", duplicate_check)
builder.add_node("classify_route", classify_route)

builder.set_entry_point("intake_clarify")
builder.add_edge("intake_clarify", "deflect")
builder.add_edge("deflect", "duplicate_check")
builder.add_edge("duplicate_check", "classify_route")
builder.add_edge("classify_route", END)

graph = builder.compile()
```

(Keep edges linear for Day 1 — real conditional branching comes Day 2 once the nodes have real logic to branch on.)

## Step 8 — `db.py` (SQLite audit log)

```
import sqlite3

def init_db():
    conn = sqlite3.connect("audit.db")
    conn.execute("""
        CREATE TABLE IF NOT EXISTS audit_log (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            session_id TEXT,
            stage TEXT,
            decision TEXT,
            confidence REAL,
            timestamp TEXT DEFAULT CURRENT_TIMESTAMP
        )
```

```
    """)
    conn.commit()
    conn.close()

def log_event(session_id, stage, decision, confidence):
    conn = sqlite3.connect("audit.db")
    conn.execute(
        "INSERT INTO audit_log (session_id, stage, decision, confidence) VALUES (?, ?, ?,
?)",
        (session_id, stage, decision, confidence)
    )
    conn.commit()
    conn.close()
```

## Step 9 — `chroma_store.py` (empty Chroma collection for B)

```
import chromadb

client = chromadb.PersistentClient(path="./chroma_data")
kb_collection = client.get_or_create_collection("kb_articles")
ticket_collection = client.get_or_create_collection("tickets")
```

## Step 10 — `main.py` (FastAPI entrypoint)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from graph import graph
from db import init_db, log_event

app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])
init_db()

class ChatRequest(BaseModel):
    session_id: str
    message: str

@app.post("/chat")
def chat(req: ChatRequest):
    state = {
        "session_id": req.session_id,
```

```
        "user_messages": [req.message],
        "extracted_fields": {},
        "missing_fields": [],
        "completeness_score": 0.0,
        "retrieved_kb_docs": [],
        "similar_tickets": [],
        "classification_confidence": 0.0,
        "routing_decision": None,
        "escalation_needed": False,
        "audit_log": [],
    }
    result = graph.invoke(state)
    log_event(req.session_id, "pipeline_complete", str(result.get("routing_decision")),
result.get("classification_confidence", 0.0))
    return result
```

Run it:

```
uvicorn main:app --reload --port 8000
```

Test it before touching the frontend at all:

```
curl -X POST http://localhost:8000/chat -H "Content-Type: application/json" -d
'{"session_id":"s1","message":"My VPN is not working"}'
```

If this returns JSON, your entire backbone is proven end-to-end.

## Step 11 — `frontend/index.html` (bare minimum, no framework)

```
<!DOCTYPE html>
<html>
<head><title>TRO Chat</title></head>
<body>
  <div id="chat"></div>
  <input id="msg" type="text" placeholder="Describe your issue..." style="width:300px">
  <button onclick="send()">Send</button>

  <script>
    const sessionId = "s_" + Math.random().toString(36).slice(2);
    async function send() {
      const input = document.getElementById("msg");
      const text = input.value;
      input.value = "";
      const res = await fetch("http://localhost:8000/chat", {
```

```
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ session_id: sessionId, message: text })
    });
    const data = await res.json();
    document.getElementById("chat").innerHTML += `<p><b>You:</b>
${text}</p><pre>${JSON.stringify(data, null, 2)}</pre>`;
  }
 </script>
</body>
</html>
```

Open it directly in a browser (double-click the file, or `python -m http.server 5500` inside `frontend/` and visit `localhost:5500`).

# End-of-Day-1 test checklist

- [ ] `uvicorn` runs without errors
- [ ] `curl` to `/chat` returns full state JSON
- [ ] SQLite file `audit.db` has a row after one request
- [ ] Chroma `./chroma_data` folder exists (empty collections are fine)
- [ ] `index.html` sends a message and displays the raw JSON response

Once all five are true, push to a shared repo — that's the exact point A and B plug their real logic into `nodes/intake_clarify.py`, `nodes/deflect.py`, and `nodes/duplicate_check.py` without needing to touch your graph or backend at all.